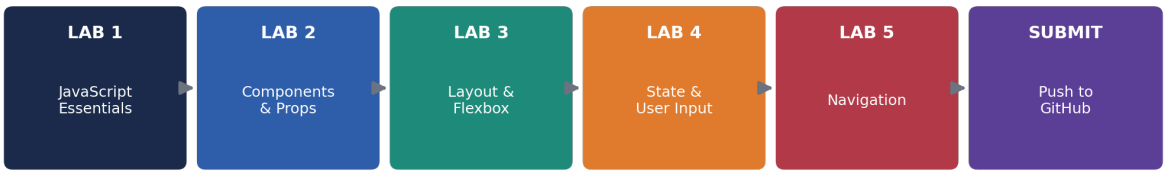
Department of Computing Education
CCE 106/L – Application Development & Emerging Technologies

Week 1 Laboratory Worksheet

Mobile App Development Crash Course — 4th Year CS-IT

5 Hands-On Labs • Real Code in VS Code • Submit via GitHub



One project, five labs, one GitHub repo your teacher can open and check

What's Inside

- Lab 1 — JavaScript Essentials for React Native
- Lab 2 — React & JSX: Components and Props
- Lab 3 — Core Components & Layout with Flexbox
- Lab 4 — State & Hooks, Handling User Input
- Lab 5 — Navigation Between Screens
- Final section — How to Submit Your Work on GitHub

NOTE: This worksheet assumes Node.js, VS Code, Android Studio/Xcode, and Expo are already installed and working on your computer. If you haven't set those up yet, complete the Environment Setup Guide first.

How to Use This Worksheet

All five labs build ONE project together, step by step, the same way the course's sample app (StudyBuddy) grows week by week. You will create the project once at the start of Lab 1, then keep adding to it through Lab 5. By the end, you'll have a small working app AND a GitHub repository your teacher can open to check your progress.

Before You Start — Checklist

- ☐ Node.js, VS Code, Android Studio (or Xcode on Mac), and Expo are installed and verified.
- ☐ You have a free GitHub account (sign up at github.com if you don't).
- ☐ Git is installed on your computer (you can check with git --version in a terminal).
- ☐ You have about 4-6 hours total to complete all 5 labs (roughly 45-60 minutes each).

How Each Lab Is Organized

- Objective — what you'll be able to do by the end.
- Setup — anything you need to install or prepare before coding.
- Steps — numbered instructions with the exact code to type into VS Code.
- Checkpoint — how to confirm it worked before moving on.
- Save Your Work — what to commit to Git before starting the next lab.

> **TIP:** Type the code yourself instead of copy-pasting when you can — typing it out is what actually builds the muscle memory. Copy-paste is fine if you're short on time, but read every line first.

🧪 LAB 1 — JavaScript Essentials for React Native

Objective: set up the one shared project for all 5 labs, connect it to GitHub, and practice the 5 JavaScript patterns React Native depends on.

Estimated time: 45-60 minutes

Setup: Create the Project

STEP 1 Create a new Expo project

Open a terminal (Command Prompt/Terminal), navigate to a folder like Desktop, and run:

```
cd Desktop
npx create-expo-app Week1Labs --template blank
cd Week1Labs
```

Open the project folder in VS Code:

STEP 2 Create a free GitHub repository

In your browser, go to github.com and log in (or sign up if you don't have an account).

Click the green New button (or the + icon top-right → New repository).

Name it Week1Labs, keep it Public, do NOT check "Add a README" (your project already has files), then click Create repository.

Keep the page open — GitHub will show you commands you'll use in the next step.

Add your teacher as collaborator (Setting → Manage Access → Add people)

[**mariceltimbal@umindanao.edu.ph**] Then click Add.

STEP 3 Connect your project to GitHub

Back in VS Code, open a terminal (Terminal menu → New Terminal) inside the Week1Labs folder and run:

```
git init
git add .
git commit -m "Initial commit: Expo project created"
git branch -M main
git remote add origin https://github.com/YOUR-USERNAME/Week1Labs.git
git push -u origin main
```

Replace YOUR-USERNAME with your actual GitHub username (copy the exact URL GitHub showed you on the repo page). Make sure that it reflects your complete name and not alias.

CHECKPOINT: Refresh your GitHub repository page in the browser — you should now see your Expo project's files there. That confirms the connection worked.

Steps: Practice the 5 JavaScript Essentials

These exercises run with plain Node.js (no emulator needed yet) so you can focus on the syntax first.

STEP 4 Create a scratch practice file

In VS Code's file explorer, right-click the Week1Labs folder and choose New File. Name it lab1-practice.js.

STEP 5 let & const

Type this into lab1-practice.js:

```
const appName = 'Week1Labs';
let taskCount = 0;
taskCount = taskCount + 1;
console.log(`${appName} has ${taskCount} task.`);
```

Run it in the VS Code terminal:

```
node lab1-practice.js
```

You should see: Week1Labs has 1 task.

STEP 6 Arrow Functions

Add this below your existing code (don't delete what's already there):

```
const double = (n) => n * 2;
console.log(`Double of 5 is ${double(5)}`);
```

Run node lab1-practice.js again. You should now see two lines printed.

STEP 7 Template Literals

You already used template literals above! Add one more example:

```
const student = 'Ana';
const streak = 3;
console.log(`${student} is on a ${streak}-day streak!`);
```

STEP 8 .map() and .filter()

Add this block:

```
const subjects = ['Math', 'Programming', 'PE', 'Networking'];
const shouting = subjects.map((s) => s.toUpperCase());
const longNames = subjects.filter((s) => s.length > 4);
console.log(shouting);
console.log(longNames);
```

Run the file. You should see two arrays printed — one all-uppercase, one with only the longer subject names.

STEP 9 Destructuring

Add this final block:

```
const task = { title: 'Finish Lab 1', done: false };
const { title, done } = task;
console.log(`Task: ${title} — Done: ${done}`);
```

Run the file one last time and confirm all 6 console.log lines print without errors.

WATCH OUT: If you see a red error in the terminal, read the error message carefully — it usually names the exact line number. Check for missing semicolons, unmatched quotes, or a typo in a variable name.

Save Your Work

STEP 10 Commit and push Lab 1

Save lab1-practice.js (Ctrl+S / Cmd+S), then in the terminal run:

```
git add .
git commit -m "Lab 1: JavaScript essentials practice"
git push
```

CHECKPOINT: Checkpoint: your GitHub repo page should now show lab1-practice.js and a commit message mentioning Lab 1. That's proof of submission for this lab.

🧪 LAB 2 — React & JSX: Components and Props

Objective: build your first real React Native component, and reuse it multiple times by passing it different props.

Estimated time: 45-60 minutes

Setup

STEP 1 Open your project and start it

In VS Code, open the Week1Labs folder from Lab 1 if it isn't already open.

Open a terminal and run:

```
npx expo start
```

Open your Android emulator (or press i on Mac for the iOS Simulator) so you can preview changes as you go.

Steps

STEP 2 Create a components folder

In the file explorer, right-click Week1Labs and choose New Folder. Name it components.

Right-click the new components folder and choose New File. Name it TaskCard.js.

STEP 3 Write your first component

Type this into TaskCard.js:

```
import { View, Text, StyleSheet } from 'react-native';

export default function TaskCard({ title, done }) {
                        return (
                    <View style={styles.card}>
                <Text style={styles.title}>{title}</Text>
                <Text>{done ? 'Done' : 'Pending'}</Text>
                        </View>
                          );
                         }

                const styles = StyleSheet.create({
card: { padding: 12, marginVertical: 6, backgroundColor: '#EEF2F8', borderRadius: 8
                                        },
```

```
  title: { fontWeight: 'bold', fontSize: 16 },
});
```

This function takes title and done as props — information passed in from wherever it's used.

STEP 4 Use your component in App.js

Open App.js and replace its entire contents with:

```
import { StyleSheet, View, Text } from 'react-native';
import TaskCard from './components/TaskCard';

export default function App() {
  return (
    <View style={styles.container}>
      <Text style={styles.heading}>My Tasks</Text>
      <TaskCard title="Finish Lab 2" done={false} />
      <TaskCard title="Read Chapter 3" done={true} />
      <TaskCard title="Walk the dog" done={false} />
    </View>
  );
}

const styles = StyleSheet.create({
  container: { flex: 1, paddingTop: 60, paddingHorizontal: 16, backgroundColor: '#FFFFFF' },
  heading: { fontSize: 24, fontWeight: 'bold', marginBottom: 12 },
});
```

Save the file. Your emulator should refresh automatically and show 3 task cards.

CHECKPOINT: Notice that the SAME TaskCard component produced 3 different-looking cards — that's the power of props. One component, reused with different data.

STEP 5 Experiment: add a 4th task

Add one more <TaskCard ... /> line in App.js with your own title and done value, then save and check the emulator.

Save Your Work

STEP 6 Commit and push Lab 2

```
git add .
git commit -m "Lab 2: Components and props with TaskCard"
git push
```

CHECKPOINT: Checkpoint: your GitHub repo should now show a components folder containing TaskCard.js, plus an updated App.js.

🧪 LAB 3 — Core Components & Layout with Flexbox

Objective: build a styled Welcome screen using View, Text, and Flexbox properties to control layout.

Estimated time: 45-60 minutes

Setup

STEP 1 Continue in the same project

Make sure Week1Labs is open in VS Code and npx expo start is still running in a terminal.

If you closed it, restart it and reopen your emulator.

Steps

STEP 2 Create a screens folder and WelcomeScreen file

Right-click Week1Labs and create a New Folder named screens.

Inside screens, create a New File named WelcomeScreen.js.

STEP 3 Build the Welcome screen layout

Type this into WelcomeScreen.js:

```
import { View, Text, StyleSheet } from 'react-native';

export default function WelcomeScreen() {
  return (
    <View style={styles.container}>
      <View style={styles.header}>
        <Text style={styles.emoji}>📱</Text>
        <Text style={styles.title}>Week1Labs</Text>
        <Text style={styles.subtitle}>Built by you, one lab at a time</Text>
      </View>
      <View style={styles.footer}>
        <Text style={styles.footerText}>Lab 3: Flexbox Layout</Text>
      </View>
    </View>
  );
}
```

```
const styles = StyleSheet.create({
  container: { flex: 1, backgroundColor: '#1B2A4A', justifyContent: 'space-between'
},
  header: { flex: 1, justifyContent: 'center', alignItems: 'center' },
  emoji: { fontSize: 64, marginBottom: 12 },
  title: { fontSize: 28, fontWeight: 'bold', color: '#FFFFFF' },
  subtitle: { fontSize: 14, color: '#C7D2E8', marginTop: 6 },
  footer: { paddingBottom: 40, alignItems: 'center' },
  footerText: { color: '#9FB0D0', fontSize: 12 },
});
```

STEP 4 Show WelcomeScreen in App.js

Open App.js and replace its contents with:

```
import WelcomeScreen from './screens/WelcomeScreen';

export default function App() {
  return <WelcomeScreen />;
}
```

Save both files. Your emulator should now show a full dark-navy welcome screen with centered content and a footer pinned to the bottom.

NOTE: flex: 1 means "take up all available space." justifyContent controls the main-axis alignment (usually vertical), alignItems controls the cross-axis (usually horizontal). Try changing justifyContent to 'flex-start' and see what happens — then change it back.

STEP 5 Experiment: change the layout

Try changing alignItems in the header style from 'center' to 'flex-start', save, and observe the change in the emulator.

Change it back to 'center' when you're done experimenting.

Save Your Work

STEP 6 Commit and push Lab 3

```
git add .
git commit -m "Lab 3: Welcome screen with Flexbox layout"
git push
```

CHECKPOINT: Checkpoint: your GitHub repo should now show a screens folder containing WelcomeScreen.js.

🧪 LAB 4 — State & Hooks, Handling User Input

Objective: use the useState hook to capture what a user types, and update what's shown on screen when they tap a button.

Estimated time: 45-60 minutes

Setup

STEP 1 Continue in the same project

Week1Labs should still be open with npx expo start running.

Steps

STEP 2 Create AddTaskScreen.js

Inside the screens folder, create a New File named AddTaskScreen.js.

STEP 3 Build a form with state

Type this into AddTaskScreen.js:

```
import { useState } from 'react';
import { View, Text, TextInput, Button, StyleSheet, FlatList } from 'react-native';
import TaskCard from '../components/TaskCard';

export default function AddTaskScreen() {
const [taskText, setTaskText] = useState('');
const [tasks, setTasks] = useState([]);

function handleAddTask() {
if (taskText.trim() === '') return;
const newTask = { id: Date.now().toString(), title: taskText, done: false };
setTasks([...tasks, newTask]);
setTaskText('');
}

return (
<View style={styles.container}>
<Text style={styles.heading}>Add a Task</Text>
<TextInput
```

```
                    style={styles.input}
                placeholder="What do you need to do?"
                      value={taskText}
                onChangeText={setTaskText}
                          />
            <Button title="Add Task" onPress={handleAddTask} />
                          <FlatList
                         data={tasks}
                keyExtractor={(item) => item.id}
renderItem={({ item }) => <TaskCard title={item.title} done={item.done} />}
                         style={styles.list}
                          />
                        </View>
                         );
                         }

                const styles = StyleSheet.create({
container: { flex: 1, paddingTop: 60, paddingHorizontal: 16, backgroundColor:
                        '#FFFFFF' },
      heading: { fontSize: 24, fontWeight: 'bold', marginBottom: 16 },
input: { borderWidth: 1, borderColor: '#D8DEE9', borderRadius: 8, padding: 10,
                      marginBottom: 10 },
                    list: { marginTop: 16 },
                          });
```

This reuses the TaskCard component you built in Lab 2 — that's the payoff of building things as reusable components.

STEP 4 Show AddTaskScreen in App.js

Open App.js and replace its contents with:

```
import AddTaskScreen from './screens/AddTaskScreen';

export default function App() {
    return <AddTaskScreen />;
                }
```

Save everything. In your emulator, type a task name, tap Add Task, and watch it appear in the list below.

CHECKPOINT: useState gives a component memory. Every time setTasks() runs, React re-draws the screen using the new array — you never manually refresh anything.

STEP 5 Experiment: add a task counter

Above the FlatList, add this line to show how many tasks exist:

```
<Text>You have {tasks.length} task(s)</Text>
```

Save and confirm the counter updates every time you add a task.

Save Your Work

STEP 6 Commit and push Lab 4

```
git add .
git commit -m "Lab 4: State, hooks, and user input with AddTaskScreen"
git push
```

CHECKPOINT: Checkpoint: your GitHub repo should now show AddTaskScreen.js inside the screens folder.

🧪 LAB 5 — Navigation Between Screens

Objective: connect WelcomeScreen and AddTaskScreen with React Navigation so users can tap between them.

Estimated time: 45-60 minutes

Setup

STEP 1 Install React Navigation

In the VS Code terminal, inside Week1Labs, run:

```
npx expo install @react-navigation/native @react-navigation/native-stack

npx expo install react-native-screens react-native-safe-area-context
```

Wait for both installs to finish before continuing.

Steps

STEP 2 Add a button to WelcomeScreen

Open screens/WelcomeScreen.js and update it to accept navigation and add a button:

```
import { View, Text, StyleSheet, Button } from 'react-native';

export default function WelcomeScreen({ navigation }) {
                    return (
                <View style={styles.container}>
                  <View style={styles.header}>
                <Text style={styles.emoji}>📱</Text>
              <Text style={styles.title}>Week1Labs</Text>
      <Text style={styles.subtitle}>Built by you, one lab at a time</Text>
                    </View>
                  <View style={styles.footer}>
                        <Button
                    title="Go to My Tasks"
            onPress={() => navigation.navigate('AddTask')}
                        />
                    </View>
                  </View>
                   );
```

```
}

const styles = StyleSheet.create({
  container: { flex: 1, backgroundColor: '#1B2A4A', justifyContent: 'space-between' },
  header: { flex: 1, justifyContent: 'center', alignItems: 'center' },
  emoji: { fontSize: 64, marginBottom: 12 },
  title: { fontSize: 28, fontWeight: 'bold', color: '#FFFFFF' },
  subtitle: { fontSize: 14, color: '#C7D2E8', marginTop: 6 },
  footer: { paddingBottom: 40, alignItems: 'center' },
});
```

The navigation prop is automatically given to every screen once it's registered in a navigator — you'll set that up next.

STEP 3 Wire up the navigator in App.js

Replace the entire contents of App.js with:

```
import { NavigationContainer } from '@react-navigation/native';
import { createNativeStackNavigator } from '@react-navigation/native-stack';
import WelcomeScreen from './screens/WelcomeScreen';
import AddTaskScreen from './screens/AddTaskScreen';

const Stack = createNativeStackNavigator();

export default function App() {
  return (
    <NavigationContainer>
      <Stack.Navigator initialRouteName="Welcome">
        <Stack.Screen
          name="Welcome"
          component={WelcomeScreen}
          options={{ headerShown: false }}
        />
        <Stack.Screen
          name="AddTask"
          component={AddTaskScreen}
          options={{ title: 'My Tasks' }}
        />
      </Stack.Navigator>
    </NavigationContainer>
  );
```

```
}
```

Save both files. Reload the app (press r in the terminal if needed). You should land on WelcomeScreen with a "Go to My Tasks" button.

STEP 4 Test the full flow

Tap "Go to My Tasks." You should slide into AddTaskScreen, now with a header bar showing "My Tasks" and a back arrow.

Add a task, then tap the back arrow to confirm you can navigate both directions.

CHECKPOINT: This is the same Stack Navigator pattern used across the whole course — every additional screen you build just needs one more <Stack.Screen> line.

STEP 5 Experiment: rename the button

Change the button's title text in WelcomeScreen.js to something of your own, save, and confirm it updates.

Save Your Work

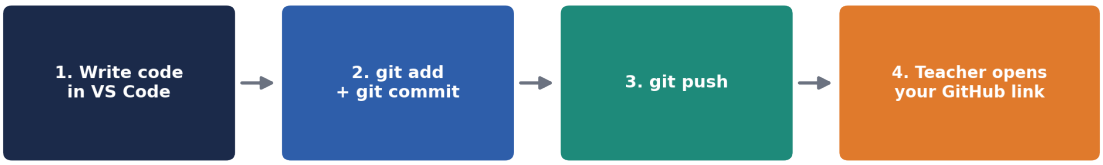
STEP 6 Commit and push Lab 5 — your final Week 1 submission

```
git add .
git commit -m "Lab 5: Navigation between Welcome and AddTask screens"
git push
```

CHECKPOINT: Checkpoint: your GitHub repo now contains all 5 labs' work in one place, with 5 separate commits telling the story of how your app grew. That's exactly what your teacher will check.

📤 Submitting Your Work on GitHub

You already connected your project to GitHub in Lab 1 and pushed after every lab since. This section is your reference for the full workflow, plus exactly what to hand in.



Repeat steps 1-3 after every lab — your commit history shows your progress

The Workflow, Explained

STEP 1 git add .

Tells Git: "include every changed file in my next save point." The dot means 'everything in this folder.'

STEP 2 git commit -m "message"

Creates an actual save point (a commit) with a short note describing what changed. Write messages a classmate could understand, like "Lab 3: Welcome screen with Flexbox layout."

STEP 3 git push

Uploads your local commits to GitHub, so they're visible online — this is the step that actually submits your work.

If Something Goes Wrong

- "git: command not found" — Git isn't installed. Download it from git-scm.com and restart your terminal.
- "fatal: not a git repository" — you're not inside the Week1Labs folder. Use cd Week1Labs first.
- "Support for password authentication was removed" — GitHub now requires a Personal Access Token instead of your password when pushing. Go to GitHub → Settings → Developer settings → Personal access tokens to create one, and use it in place of your password when prompted.
- git push says "rejected" — someone (or GitHub itself) changed something remotely. Run git pull first, then try git push again.

How Your Teacher Will Check Your Work

Your teacher only needs your repository's link. They will open it in a browser and expect to see:

- ☐ A repository named Week1Labs (or similar), set to Public.
- ☐ 5 commits in the history, one per lab, with readable messages.
- ☐ A components folder containing TaskCard.js.
- ☐ A screens folder containing WelcomeScreen.js and AddTaskScreen.js.
- ☐ An App.js that wires everything together with navigation.
- ☐ lab1-practice.js showing your JavaScript essentials practice.

How to Share Your Repository Link

STEP 4 Copy your repository URL

On your GitHub repository page, click the green Code button and copy the HTTPS link (it looks like https://github.com/your-username/Week1Labs).

STEP 5 Submit the link

Paste this link wherever your teacher asks for it — your LMS, a shared form, or email. Double-check the link opens correctly in a private/incognito browser window before submitting, so you know it's really public.

WATCH OUT: A repository set to Private cannot be viewed by your teacher unless you specifically add them as a collaborator. When in doubt, keep class assignment repositories Public.

Final Submission Checklist

- ☐ Lab 1: JavaScript essentials practice file committed and pushed
- ☐ Lab 2: TaskCard component built and reused with different props
- ☐ Lab 3: WelcomeScreen laid out with Flexbox
- ☐ Lab 4: AddTaskScreen working with useState and TextInput
- ☐ Lab 5: Navigation working between Welcome and AddTask screens
- ☐ Repository is Public and the link has been tested in an incognito window
- ☐ Teacher is added as collaborator of the Repository

CHECKPOINT: Congratulations — you've completed Week 1! You now have a small working app, five GitHub commits telling the story of how you built it, and every JavaScript and React Native pattern you'll keep using for the rest of the course.